



A continuación encontrará un formato para la versión final de su texto listo para publicación, si tiene alguna duda puede consultar en los términos de referencia de la convocatoria pública

(INDICACIONES DEL TEXTO: ARIAL, tamaño 12; con interlineado 1,5, texto justificado; máximo 8000 palabras; el documento no debe tener secciones adicionales a las presentes en este documento o tablas de contenido; las tablas, imágenes, ilustraciones o cuadros deben estar centrados y debidamente identificados con título por encima y fuente por debajo; normas APA 7.^a edición.

Toda fotografía, imagen, figura o gráfico debe estar soportada por un texto descriptivo con fuente origen.

Diseño e Implementación de un Sistema Automatizado para la Generación de Cotizaciones Comerciales en HTML con Python

Daniel García Araque¹, Milton Jimenez Moreno

(A continuación, no borre el enunciado, luego de los dos puntos (:) deje un espacio y coloque las respuestas en formato mayúsculo en letra inicial de cada palabra)

Nombre del semillero: Semillero de Desarrollo de Software y Automatización

Área del conocimiento: Ingeniería y Tecnología

Líneas de investigación: Ingeniería de Software; Automatización de Procesos; Transformación Digital

Grupo de investigación:

Sector o sectores impactados por el proyecto: Servicios empresariales, comercio digital y pymes

¹ Estudiante , est.daniel.garciaa@unimilitar.edu.co

Entidades participantes en el proyecto: Empresa desarrolladora del sistema y organizaciones usuarias del módulo de cotizaciones

Evento en el que participó y año:

RESUMEN Este trabajo presenta el diseño e implementación de un sistema automatizado para la generación de cotizaciones comerciales en formato HTML, desarrollado con Python y orientado a optimizar la elaboración de propuestas para clientes. El problema abordado corresponde a los tiempos elevados, la inconsistencia documental y la alta probabilidad de errores manuales en procesos de cotización realizados de forma artesanal. Como objetivo principal se definió construir una solución que integrara captura estructurada de datos, selección dinámica de servicios, parametrización de precios y generación de salidas listas para publicación o entrega digital.

La metodología incluyó análisis funcional del flujo comercial, modelado de variables críticas, construcción iterativa de scripts en consola y una interfaz gráfica con PyQt6, seguida de validación técnica mediante pruebas manuales de escenarios representativos. El sistema resultante permite seleccionar servicios (desarrollo web, redes sociales, bot de WhatsApp, campañas de Facebook y capacitación en IA), calcular valores comerciales, aplicar descuentos y producir una cotización personalizada con estructura visual estandarizada. Como resultados, se evidenció reducción del retrabajo operativo, mayor trazabilidad en la información y mejora en la consistencia del documento final.

El proyecto demuestra que una arquitectura ligera basada en plantillas HTML y lógica de negocio en Python puede resolver necesidades reales de automatización comercial en organizaciones con requerimientos de respuesta rápida y presentación profesional.

PALABRAS CLAVE

Automatización; Cotizaciones; Python; HTML; PyQt6

INTRODUCCIÓN

La digitalización de procesos comerciales exige herramientas que permitan responder con agilidad y precisión a las solicitudes de clientes. En múltiples organizaciones, la creación de cotizaciones continúa realizándose mediante edición manual de documentos, lo que genera demoras, duplicidad de trabajo y variabilidad en la calidad de la presentación final. Esta situación problemática afecta la eficiencia operativa y la percepción de profesionalismo frente al cliente, especialmente en contextos donde la velocidad de respuesta influye en la conversión comercial.

En el caso analizado, la elaboración de propuestas para servicios digitales implicaba consolidar datos de cliente, listar servicios ofertados, calcular precios y adaptar el formato visual de manera repetitiva. Entre las causas directas se identificaron la ausencia de un flujo estandarizado de captura de información, la falta de reutilización estructurada de plantillas y la dependencia de tareas manuales para ajustar contenidos y valores. Como causas indirectas se reconocieron limitaciones en prácticas de validación de datos y baja integración entre componentes de captura, cálculo y salida documental.

Si esta problemática no se interviene, sus consecuencias incluyen incremento de errores en precios y textos, inconsistencias entre versiones de cotizaciones, mayores tiempos de ciclo comercial y reducción de la capacidad de escalamiento del proceso. En respuesta, se planteó el desarrollo de un sistema automatizado que centraliza la lógica del proceso y produce documentos HTML homogéneos y personalizables.

DISEÑO METODOLÓGICO

La ruta metodológica se estructuró en cinco fases: diagnóstico, diseño funcional, implementación, validación y ajuste. En la fase de diagnóstico se levantaron los requerimientos de negocio asociados al proceso de cotización, identificando entradas, reglas y resultados esperados. Posteriormente, en la fase de diseño funcional se definieron las entidades principales del sistema (cliente, empresa, servicios, precios, términos de oferta) y sus relaciones con la salida documental.

La implementación se desarrolló en Python bajo un enfoque incremental. Se construyó una versión de consola para consolidar la lógica base de captura y validación, y luego una interfaz gráfica con PyQt6 para mejorar la usabilidad y reducir errores de digitación. La generación del documento se resolvió con plantilla HTML y sustitución de variables mediante `string.Template`, permitiendo activar o desactivar bloques de contenido según servicios seleccionados.

Como escenarios de actuación, el sistema se probó en la generación de propuestas con combinaciones diversas de servicios, en validación de entradas numéricas de precios y en construcción de estructura de carpetas por cliente para facilitar entrega y publicación. Los aliados principales fueron el equipo de desarrollo del sistema y los usuarios internos responsables del proceso comercial, quienes aportaron retroalimentación para los ajustes de interfaz y flujo.

RESULTADOS E IMPACTOS:

El sistema implementado permitió estandarizar la producción de cotizaciones comerciales y disminuir la variabilidad del documento final. Entre los resultados observados se destaca la automatización de tareas repetitivas en captura de datos, cálculo de valores y ensamblaje del contenido HTML, así como la generación de archivos organizados por cliente para procesos de entrega.

En términos de impacto, se identificó mejora en tres dimensiones: eficiencia operativa, calidad documental y trazabilidad. En eficiencia operativa, el flujo asistido por interfaz gráfica y validaciones redujo tiempos de elaboración y corrección. En calidad documental, la plantilla única evitó diferencias estructurales entre propuestas y fortaleció la presentación institucional. En trazabilidad, la separación por carpetas y la parametrización de servicios facilitaron control de versiones y revisión de salidas.

Adicionalmente, la incorporación de validación de precios y manejo dinámico de secciones disminuyó errores comunes relacionados con formatos numéricos y contenido no pertinente. En conjunto, los hallazgos permiten concluir que la solución desarrollada es suficiente y coherente para soportar procesos reales de cotización en organizaciones que ofrecen servicios digitales, con potencial de ampliación hacia pruebas automáticas y analítica de desempeño comercial.

REFERENCIAS

American Psychological Association. (2020). Publication manual of the American Psychological Association (7th ed.). American Psychological Association.

Beck, K., & Andres, C. (2004). Extreme programming explained: Embrace change (2nd ed.). Addison-Wesley.

Boehm, B. W. (1988). A spiral model of software development and enhancement. *Computer*, 21(5), 61-72. <https://doi.org/10.1109/2.59>

Brooks, F. P. (1995). The mythical man-month: Essays on software engineering (Anniversary ed.). Addison-Wesley.

Fowler, M. (2018). Refactoring: Improving the design of existing code (2nd ed.). Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Hunt, A., & Thomas, D. (2019). The pragmatic programmer: Your journey to mastery (20th anniversary ed.). Addison-Wesley.

ISO/IEC/IEEE. (2017). ISO/IEC/IEEE 12207:2017 systems and software engineering - software life cycle processes. ISO.

Larman, C. (2004). Agile and iterative development: A manager's guide. Addison-Wesley.

Martin, R. C. (2009). Clean code: A handbook of agile software craftsmanship. Prentice Hall.

McConnell, S. (2004). Code complete (2nd ed.). Microsoft Press.

Meyer, B. (1997). Object-oriented software construction (2nd ed.). Prentice Hall.

Nielsen, J. (1994). Usability engineering. Morgan Kaufmann.

Petersen, K., Vakkalanka, S., & Kuzniarz, L. (2015). Guidelines for conducting systematic mapping studies in software engineering. Information and Software Technology, 64, 1-18. <https://doi.org/10.1016/j.infsof.2015.03.007>

Pressman, R. S., & Maxim, B. R. (2019). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill.

Project Management Institute. (2021). A guide to the project management body of knowledge (PMBOK guide) (7th ed.). Project Management Institute.

Python Software Foundation. (2024). Python documentation.
<https://docs.python.org/3/>

PyQt. (2024). PyQt6 reference guide.
<https://www.riverbankcomputing.com/static/Docs/PyQt6/>

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

Sutherland, J. (2014). Scrum: The art of doing twice the work in half the time.
Crown Business.

W3C. (2018). HTML living standard. <https://html.spec.whatwg.org/>

Wiegers, K. E., & Beatty, J. (2013). Software requirements (3rd ed.). Microsoft Press.

Yourdon, E. (1989). Modern structured analysis. Prentice Hall.

Bootstrap Team. (2024). Bootstrap documentation. <https://getbootstrap.com/docs/>

Meta. (2024). Meta business help center. <https://www.facebook.com/business/help>

OpenAI. (2024). Prompt engineering best practices.
<https://platform.openai.com/docs>

Pylance Team. (2024). Python language support in Visual Studio Code.
<https://code.visualstudio.com/docs/languages/python>

Reeves, S., & Zhu, Y. (2020). Practical software testing: A process-oriented approach. Springer.

Riverbank Computing. (2024). Qt for Python and desktop interface patterns.
<https://www.riverbankcomputing.com/>

Swebok Guide. (2014). Guide to the software engineering body of knowledge (SWEBOK v3.0). IEEE Computer Society.

ANEXOS